

1) Definition Softwaretechnik

Der Begriff Softwaretechnik leitet sich aus der Software mit aktiven und passiven Daten ab. Aktive Daten sind dabei z. B. Programme und passive Daten können unter anderem Anwenderdokumentationen sein. Der andere Begriff ist die Technik und kommt aus dem Ingenieurwesen. Hierbei geht es in der Softwaretechnik um das praktische Anwenden von Fachwissen.

Softwaretechnik ist also die systematische und ingenieurmäßige Entwicklung, der Betrieb sowie die Wartung von Software. Sie umfasst den gesamten Softwarelebenszyklus – von der ersten Idee über Requirements Engineering, Analyse, Entwurf und Implementierung bis hin zur Wartung und Pflege.

Software besteht dabei nicht nur aus Code, sondern auch aus Dokumentation, Tests und weiteren Artefakten. Softwaretechnik verbindet technische, organisatorische und methodische Aspekte, um Software reproduzierbar und qualitativ hochwertig zu entwickeln.

Ein zentrales Merkmal der Softwaretechnik ist die Beherrschung von Komplexität durch strukturierte Methoden, Modelle und Prinzipien. Gerade bei großen Systemen ist dies entscheidend, um Fehlentwicklungen zu vermeiden und langfristige Wartbarkeit sicherzustellen.

2) Softwarequalität

Softwarequalität beschreibt für mich den Grad, in dem ein Softwaresystem die definierten Anforderungen erfüllt und gleichzeitig zuverlässig, effizient, sicher und wartbar ist. Sie ist ein zentrales Ziel der Softwaretechnik und wird durch kontinuierliche Qualitätssicherung während des gesamten Entwicklungsprozesses erreicht. Ich denke dass die Benutzerfreundlichkeit und die Sicherheit in allen Bereichen für mich hier die zentralen Punkte sind. Nur wenn die Daten sicher sind und zum Beispiel vor unerlaubten Zugriffen von außen geschützt sind und ich bei Problemen mit der Software schnell und unkompliziert mir entweder selbst helfen kann, oder mir Hilfe organisieren kann, so ist die Software gut nutzbar. Mit regelmäßigen Updates und Patches sollte eine ständige Aktualisierung erreicht werden.

Softwarequalität ist sehr wichtig, da Fehler in der Software riesige Schäden anrichten, wie in dem Beispiel mit der Rakete erläutert im Material. Eine schlechte Qualität führt zu hohen Wartungskosten. Die Nutzerzufriedenheit und das Vertrauen hängen direkt von der Qualität ab. Qualität ist daher kein Endzustand, sondern ein fortlaufender Prozess innerhalb des gesamten Softwarelebenszyklus.

3) Hauptprobleme von Softwareprojekten und Lösungen

Unklare oder unvollständige Anforderungen:

→ Lösung: strukturiertes Requirements Engineering, klare Dokumentation, kontinuierliche Validierung mit Stakeholdern

Schlechte Kommunikation und mangelnde Einbindung:

→ Lösung: regelmäßige Abstimmungen, Feedback-Schleifen, transparente Prozesse

Menschliche Befindlichkeiten (aus meiner eigenen Erfahrung), wie z. B. Vorbehalte vor Veränderungen, Angst vor Statusverlusten

→ Lösung: sehr gute Kommunikation und “abholen” aller Beteiligten, Erläuterungen des was, warum, wie und der Zukunft

Mangelnde Planung und Ressourcenmanagement:

→ Lösung: realistische Zeitplanung, klare Meilensteine, iterative Entwicklung (Best Practices wie kleine Schritte), hierbei meiner Meinung nach besonders wichtig ist die ständige schriftliche Dokumentation von Meilensteinen, Zwischenzielerreichungen und allen relevanten Daten und Informationen

Auffällig ist, dass viele dieser Probleme nicht primär technischer Natur sind, sondern organisatorische und kommunikative Ursachen haben. Dies zeigt, dass erfolgreiche Softwareentwicklung immer auch ein Management- und Kommunikationsproblem ist.

4) Softwarelebenszyklus nach Balzert Analyse → Definition → Entwurf → Implementierung → Abnahme/Einführung → Wartung/Pflege

Analysephase: Ausgangssituation, Ziele und Wirtschaftlichkeit

Definitionsphase: Anforderungen festlegen

Entwurfsphase: Architektur und Systemdesign

Implementierungsphase: Umsetzung in Code und Tests

Einführungsphase: Installation und Nutzung

Wartungsphase: Fehlerbehebung und Weiterentwicklung

In der Praxis werden diese Phasen häufig iterativ durchlaufen und sind nicht strikt linear.

5) Persönlicher Bezug und Interessen

Ich durfte vor 2 Jahren in einer Firma 4 Wochen an einem iterativen Prozess teilnehmen um Software für Gesundheitsämter auf dem Laufenden zu halten, Probleme zu beheben und ein Update einzuspielen. Das war sehr interessant und besonders überrascht hatte mich der große Aufwand dies auch Alles zu dokumentieren und in vielen einzelnen Tickets zu bearbeiten und zu kommunizieren.

Aktuell arbeite ich im ITDZ-Berlin AöR in der Auftragssteuerung. Hier wird es bald einige wichtige Wechsel von Softwareprodukten geben. Zum Beispiel geht es konkret um einen Übergang der Auftragssteuerung von SAP R/3 zu dem Softwareprodukt "ServiceNow".

Hierfür und natürlich auch allgemein erhoffe ich mir, mich noch mehr mit den Prozessen und den Best Practices der Softwaretechnik in Zukunft auszukennen und diese anwenden zu können.

Und als Student der Wirtschaftsinformatik beschäftige ich mich intensiv mit der Verbindung zwischen fachlichen Anforderungen und technischer Umsetzung. Softwaretechnik hilft mir, komplexe Systeme strukturiert zu analysieren, zu planen und umzusetzen.

Besonders interessieren mich:

- Requirements Engineering
- Softwarearchitektur und UML
- Versionsmanagement und Refactoring
- Cloud Computing
- Anwendungen hier in Zusammenarbeit mit AI/LLM

Weiterführende Links:

<https://www.omg.org/uml/>

<https://git-scm.com/docs>

6) Einfluss von AI / LLM auf die Softwaretechnik

AI und Large Language Models beeinflussen die Softwaretechnik schon jetzt stark und dies wird weiter zunehmen. Den Menschen an sich, wird dies meiner Meinung nach, aber nie vollständig ersetzen können. Bisher gibt es zum Beispiel bereits Beeinflussungen durch:

- Unterstützung bei der Programmierung und Code-Generierung
- Automatisierte Erstellung von Dokumentation und Artefakten
- Analyse von Systemen, Anforderungen und Fehlern
- Automatisierung von Routineaufgaben

Die Rolle des Entwicklers verändert sich dabei vom reinen Programmierer hin zum Orchestrator, der Systeme versteht, bewertet und steuert. Auch ist aktuell noch eine Fehlerquote der AI vorhanden, welche unbedingt beachtet und die Fehler bearbeitet werden müssen. Das reicht von "Halluzinationen" der LLMs über grobe Rechenfehler und auch z. B. Formatierungs- und Annahmefehlern der AI und vieles mehr.

Was gut und sehr schnell die AI übernimmt und noch mehr übernehmen wird sind z. B. Programmierung von Code, welcher dann noch korrigiert werden muss, Erstellen von Codes für Prototypen. Schnelle Abwandlung von vorhandenem Code um Modelle für Versionen zu erstellen. Bug-Suche und Behebung. Testing. Wartung, Anwendungsdokumentation im Code und in Handbücher und viele andere Dinge.

In ca. 5-10 Jahren kann ich mir vorstellen, dass AI auch eigene Softwaretechniklösungen selbst entwickelt, programmiert und anwenden wird.

Langfristig könnte sich dadurch ein Paradigmenwechsel ergeben: weg von reiner Implementierung hin zu stärker modellgetriebener und konzeptioneller Softwareentwicklung, bei der das Verständnis von Anforderungen und Systemarchitektur entscheidender wird als das reine Schreiben von Code.